APPLIED MATH 50 LAB 2: IMAGING
MATRICES, IMAGES, AND THE RADON TRANSFORM

1. Introduction

Today's computer lab has two aims:

(1) Learn how to generate, import, and manipulate images (matrices) in Matlab
(2) Explore the Radon transform and applications to medical image reconstruction.

2. Getting started with images

We'll begin this lab by discussing how to handle images in Matlab. Matlab has good support for manipulating images via the image processing toolbox. To find out of this toolbox is installed on your computer, type `ver` at the prompt in the command window and hit enter. Scroll through the list and make sure that you see "Image Processing Toolbox." If you do, then you're ready to get started.

As we briefly described in class 8, an image can be represented as a matrix. For grayscale image, the entries v_{ij} of the matrix represent the pixel intensity, with $v_{ij} = 1$ corresponding to white, $v_{ij} = 0$ corresponding to black, and $0 < v_{ij} < 1$ corresponding to various shades of gray. Let's examine the code for the simple 81 pixel image generated by the script file below.

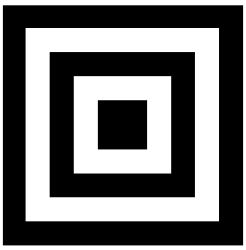
Exercise 1
Read through the code. On paper, carefully draw the 81 pixel image according to the commands defined in the script.

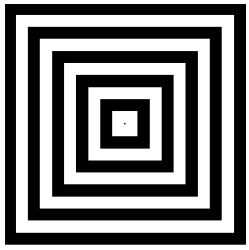
```
clear all;
close all;

M = 9;
N = 9;

%Creates an M by N matrix of zeros (black)
V = zeros(M,N);

%Change certain entries to white
V(3,3) = 1;
V(3,7) = 1;
V(5,5) = 1;
V(6,2) = 1;
V(6,8) = 1;
V(7,3) = 1;
V(7,7) = 1;
V(8, 4:6) = 1;
```

```
%Display the image
imshow(V, 'InitialMagnification', 'fit')
```

Type the script into Matlab and save it as `SimpleImage.m`. Run the script and compare the output to your sketch in Exercise 1. If your sketch doesn't agree with the Matlab output, go back to your sketch to reconcile the differences.

Now let's create the 200 x 200 pixel images below.





Open and save a new script called `BigSimpleImage.m` and make a 200 x 200 black image. Then write and save a new function called `makesquare.m`, according the instructions in the exercise 2 below.

Exercise 2

Write a function that takes in an image and outputs the same image with a square of a specified shade (black or white). The function should have six inputs: the image, the coordinates of the pixel that corresponds to the upper left corner of the square, the dimensions of the square, and the shade of the square.

Exercise 3

Call your function several times in BigSimpleImage.m to create the leftmost image above. Then modify your script to make the rightmost image. For the rightmost image, you may want to use a loop as well as the Matlab function mod.

3. Color images and real images

Color images can be created by using three separate channels for the red, green, and blue components. To represent an image of M rows and N columns in Matlab, a three dimensional array `V` of size $M \times N \times 3$ can be created with `V(i, j, 1)`, `V(i, j, 2)`, and `V(i, j, 3)` representing the red, green, and blue channel values, respectively. An arbitrary color can be made by combining these three components.

Exercise 4

Modify the script from exercise 1 to make a color image with blue eyes, red nose, and orange mouth. See https://www.w3schools.com/colors/colors_rgb.asp for tips on making different colors.

Now let's play around with a real image. To do this we first need a real image to import into Matlab. Go to my.harvard and download and save your house photo as a headshot.jpg in your working directory. In the command window, type the following:

```
clear all;
I = imread('headshot.jpg');
save I
```

The Matlab command `imread` reads in headshot.jpg and stores it in the matrix `I`. Then command `save` saves the matrix `I` as `I.mat`. If you type `ls` in your command window, all the files in your working directory are listed. You should see the file `I.mat` which contains the variable `I`. Now open and save a new script `MyHousePhoto.m` with the following lines of code:

```
clear all;

%load the variable I that we saved above as I.mat
load I

%display image
imshow(I);

%change image from color to black-and-white
Ibw = rgb2gray(I);
%display black-and-white image
imshow(Ibw)

%convert the entries in Ibw to double
Ibw = im2double(Ibw);
```

The command `Ibw = im2double(Ibw)` converts every entry in the image matrix from "unsigned 8-bit integer" to "double." If you're not familiar with these data types, don't worry about it. Images tend to be very large and require a lot of computer memory for storage, and storing images as unsigned 8-bit integer (uint8) saves space. But Matlab doesn't like to operate on uint8 matrices, so we need to first convert the entries of the image matrix to a format that Matlab likes, ie, double. You can then modify the image in whatever way you like and save it by first converting back to uint8 with the command `im2uint8`.

Exercise 5

Crop your house photo and make a new color image with only your eyes. You can crop an image by estimating the coordinates the region of interest, or you can use a Matlab function called imcrop. Then modify the image so that your left eye is on the right hand side of the image and your right eye is on the left hand side.

4. Transforms and image reconstruction

Last week in class we discussed inverse problems, particularly in the context of radiology and CT-scans. Suppose we're given a data set and we're told that it contains information

about an image. Can we recover the image from this data? A CT scanner acquires data about the attenuation distribution of the materials in your body. Then – without cutting you open! – a computer reconstructs an image of your insides by processing the data in a special way, ie, by inverting the Radon transform. We'll explore this with a simple image. Make a new script called `ImageRecon.m` and create a 128 x 128 image with an 32 x 32 white square in the center of the image. The square represents you, about to undergo a CT scan. The command

```
sinogram =  radon(image, angles)
```

takes the Radon transform of the image at given angles with respect to the horizontal. We began talking about this magical transform in class, and we'll continue to work with it on next week's homework. The important point is that a CT scanner essentially measures the Radon transform of you, taken from various views around your body. For example, `sinogram =  radon(image, 0:180:360)` takes the Radon transform of the image matrix at $\theta = 0, 180, 360$ degrees. Add this command to your script. Then run the script and notice the size of the variable `sinogram`. The first column is the Radon transform "shadow" of the image taken along horizontal lines. This shadow is often called a projection. The second column is also a projection, this time taken along lines that form 180^o angles with respect to the horizontal. The third column is the projection along lines that form 360^o angles with respect to the horizontal. And so on.

Exercise 6

Use the Matlab function subplot to plot the first, second, and third columns of sinogram on one figure but in three separate plots. Do these projections make sense given what you know about you? (Remember that for the purposes of this exercise, you are a square.)

You can get the projections at many more angles by modifying the angle argument in `radon`. Modify the command in your script to calculate the Radon transform at $\theta = 0, 4, 8, \ldots, 360$.

Exercise 7

Since you are an object with symmetry, you might suspect that some of your projections are identical. When two projections are identical, we say that they are redundant. Determine which projections are redundant.

Now given that you only know the projections (`sinogram` in your code) from the scanner, how do we reconstruct your original image of your insides (`image` in your code)? The command

```
imagerecon = iradon(sinogram, angles)
```

inverts the Radon transform to recover the original image!

Exercise 8

Modify your script to compute the Radon transform only at $\theta = 0, 180, 360$. Then use iradon to invert the transform. Use subplot to make a figure with two images: the original image and the reconstructed image. Does your reconstructed image look like the original image? What goes wrong? Try:

(1) Using more angles: calculate the Radon transform at $\theta = 0, 4, 8, \ldots, 360$. Plot the original image and the reconstructed image.
(2) Using more angles but only between $\theta = 0$ and $\theta = 180$. Calculate the Radon transform at $\theta = 0, 1, 2, \ldots, 180$. Plot the original image and the reconstructed image. Does the reconstructed image look better, worse, or the same as the reconstructed image in (1)?